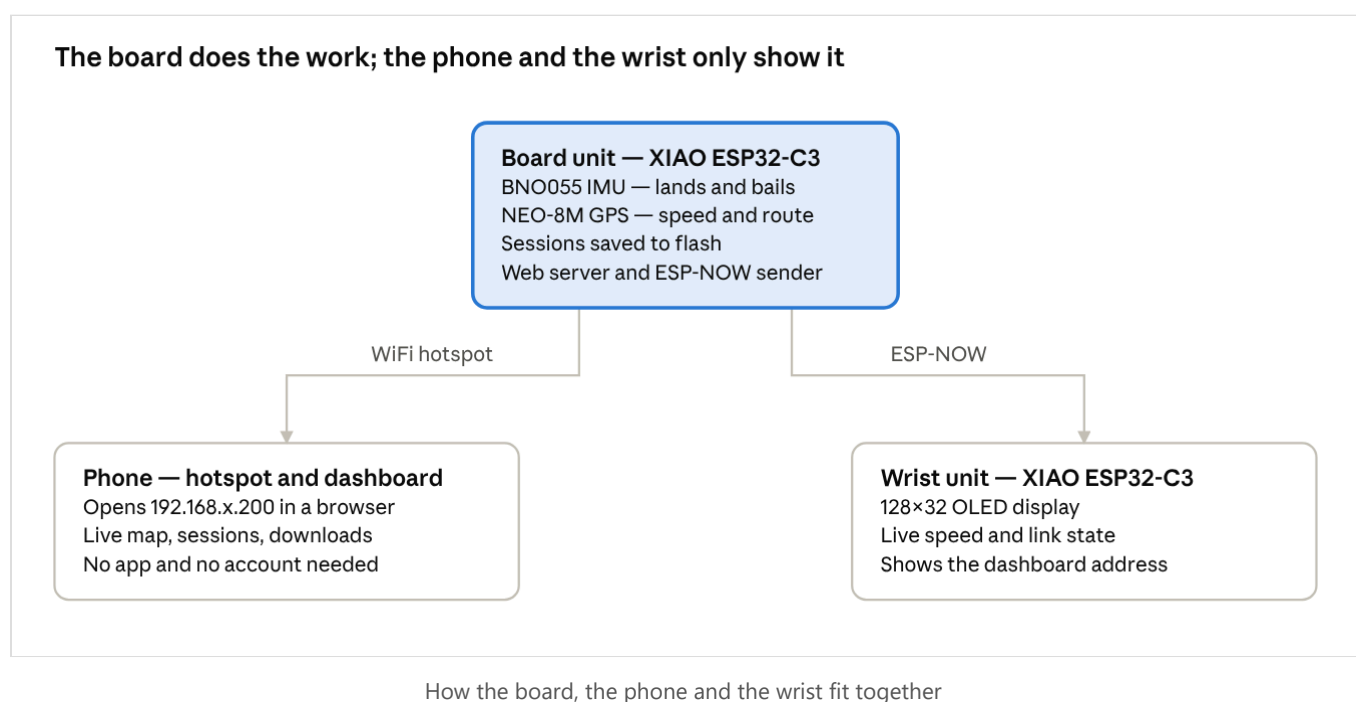


SkateGuard — 6-Week Project Report

2026-10-03 · Nikhil Merude

SkateGuard counts kickflip lands and bails, tracks speed, distance and route, and serves its own dashboard from the board to the rider's phone over a hotspot, with a wrist unit showing live speed and the dashboard address. Weeks 1 to 5 were delivered in full and Week 6 is under way, with four design changes made during the build: a BNO055 IMU in place of the MPU6050, ESP-NOW plus a self-hosted web dashboard in place of BLE and a companion app, oscillation-based wobble detection in place of a 25 km/h speed trigger, and bails judged by how the deck settles rather than by an impact spike. This report follows the plan week by week.

At a glance



The board is the only part that senses, decides and stores; the phone and the wrist are displays. Nothing leaves the rider's hotspot, so the system works with no mobile data and no account.

The six weeks at a glance

Every week's objective was met, several by a different route than planned. Changes from the original plan, near the end of this report, sets out each change and why it was made.

Week	Planned	Delivered
1	Finalise layout, lock the wobble threshold at 25 km/h, confirm the Kickflip approach, procure ESP32, MPU6050, GPS, BLE wearable parts, motor, battery	Layout fixed: board unit under the deck, wrist unit on the strap. Parts bought around two XIAO ESP32-C3 boards, a BNO055 IMU and a NEO-8M GPS. Wobble redefined as an oscillation pattern rather than a speed trigger
2	Mechanical layout, 3D-printed board housing and wrist casing, ESP32 toolchain, basic IMU and GPS communication	Both enclosures built. Arduino IDE with ESP32 core 3.2.0 on XIAO_ESP32C3, IMU reading over I2C and GPS over UART1 at 9600 baud
3	Firmware reading live IMU and GPS, wobble algorithm with the 25 km/h gate, Kickflip detection from rotational signatures	Main loop reading both sensors continuously. Kickflip detection built as a four-stage state machine on the gyro's Y axis. Wobble detection built on counted oscillations, with no speed gate
4	BLE between board and wearable, haptic feedback on wobble, bail detection from impact spikes, land versus bail classification	ESP-NOW link between the two boards, with channel hunting. Land and bail separated by how the deck settles after the rotation, not by an impact spike
5	Companion app or dashboard, GPS route mapping, ride statistics, post-ride summary, live sync over BLE	Dashboard served by the board itself over the phone hotspot: live readout, route map, session list, per-session CSV and route-map download
6	Full system testing across speeds and surfaces, tune sensitivity, finalise accuracy, final assembly, demonstration and documentation	Firmware, dashboard and both enclosures assembled and running. Bench and browser testing complete; on-road tuning of detection sensitivity is in progress

Phase 1 — Foundation and Design (Week 1)

Planned: finalise the prototype layout, lock the wobble threshold, confirm the kickflip approach and procure the hardware.

Delivered: the layout and the detection approach were fixed, and the parts list was bought around two XIAO ESP32-C3 boards.

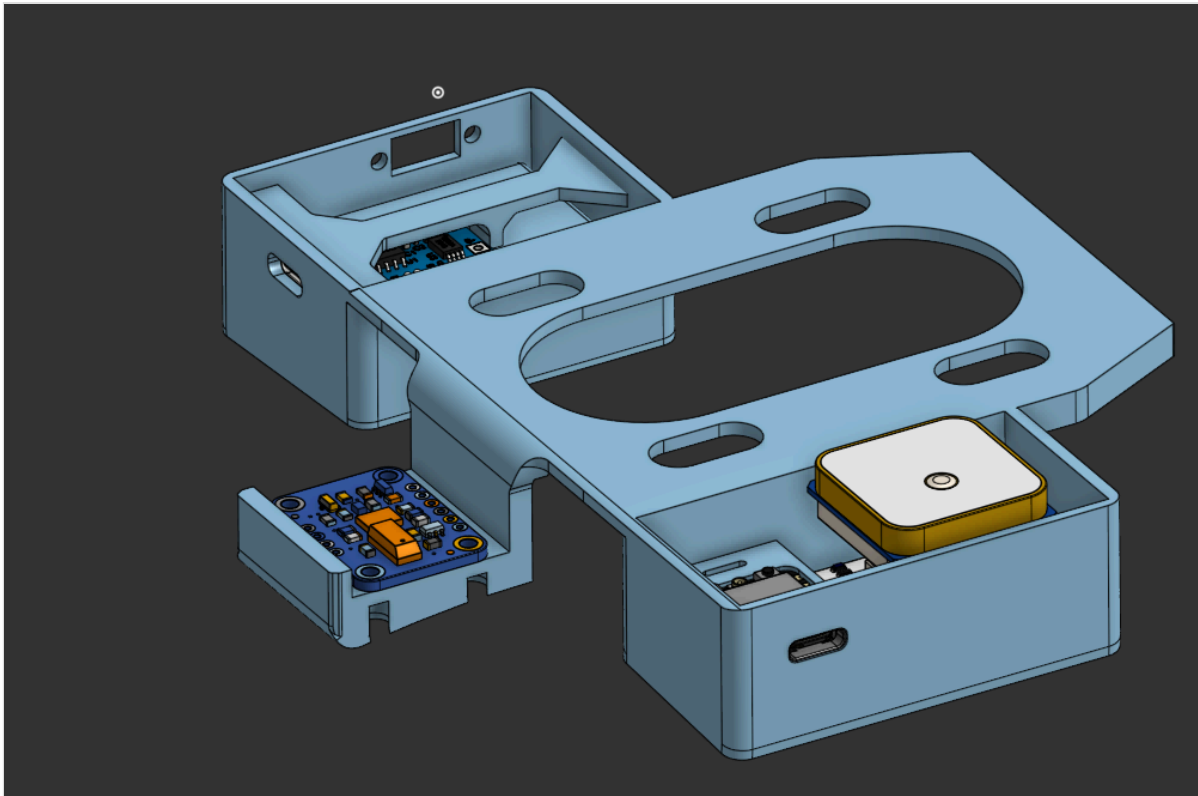
- Board unit mounted under the deck on a plate that bolts around the truck baseplate; wrist unit on a strap.
- Parts: two Seeed XIAO ESP32-C3 boards, a BNO055 IMU, a NEO-8M GPS, an SSD1306 128×32 OLED, two 1500 mAh lithium cells, two USB-C charging modules, a vibration motor and indicator LEDs.
- Kickflip detection scoped to rotation on one gyro axis, with the landing judged by how the deck settles rather than by an impact spike.
- Wobble redefined as a counted oscillation pattern instead of a 25 km/h speed trigger. The reasoning is in the root README and the project report.

CAD design

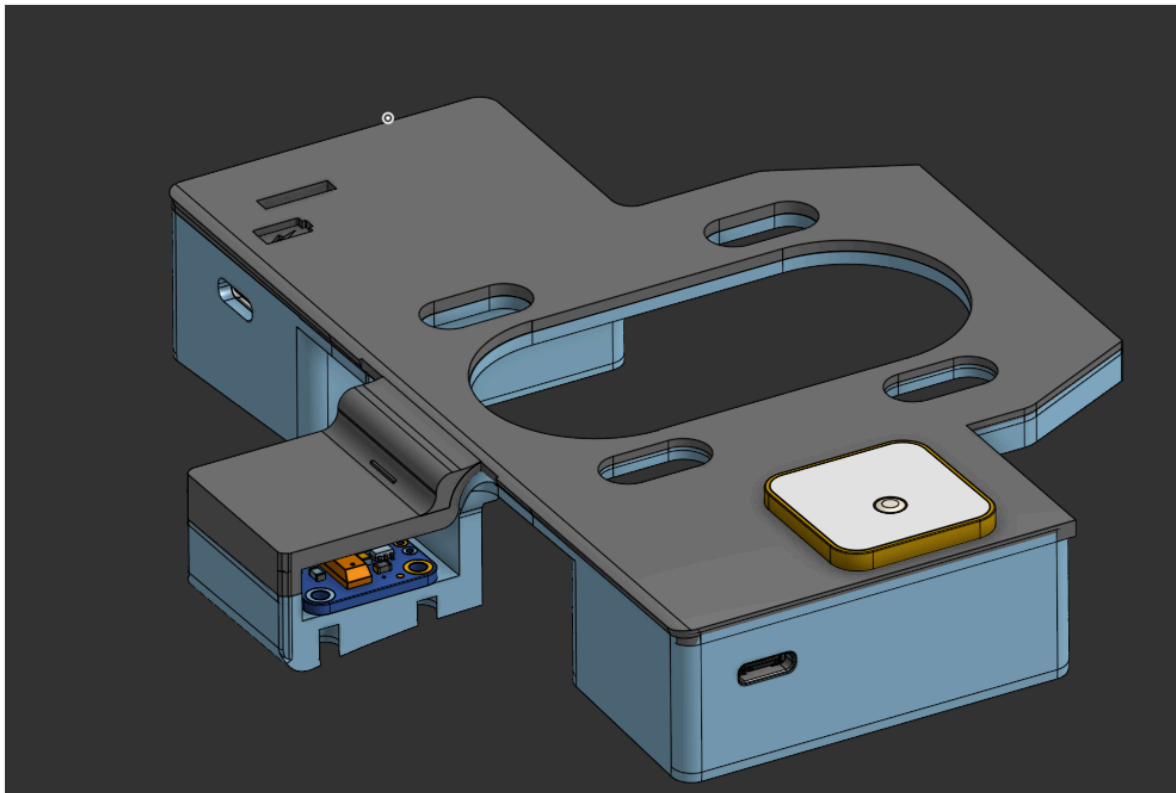
Both enclosures were designed in CAD before anything was printed, so component clearances, port positions and the mounting pattern were settled on screen rather than in filament. Both are 3D-printed, two-part enclosures holding their board, a 1500 mAh lithium cell and a USB-C charging module, with every port and indicator reachable without opening the case.

Board unit

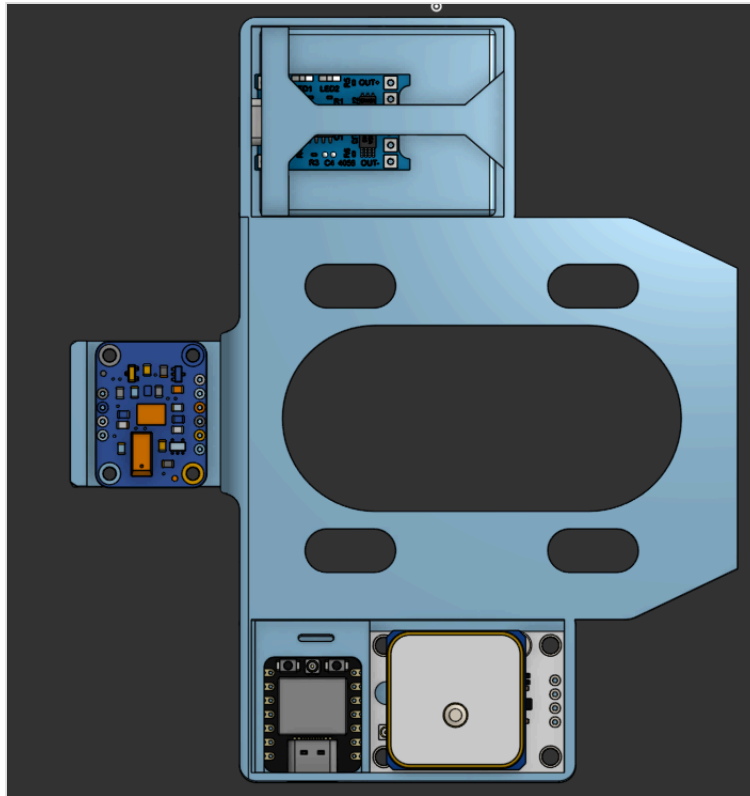
The board enclosure is a mounting plate that sits between the deck and the truck, with the hardware split into three compartments so each part gets what it needs: the GPS patch antenna faces up with a clear view of the sky, the IMU sits in its own pod on the plate's edge where it reads the deck's rotation directly, and the XIAO, cell and charging module fill the opposite compartment. The oval and slotted cut-outs drop weight and clear the mounting bolts.



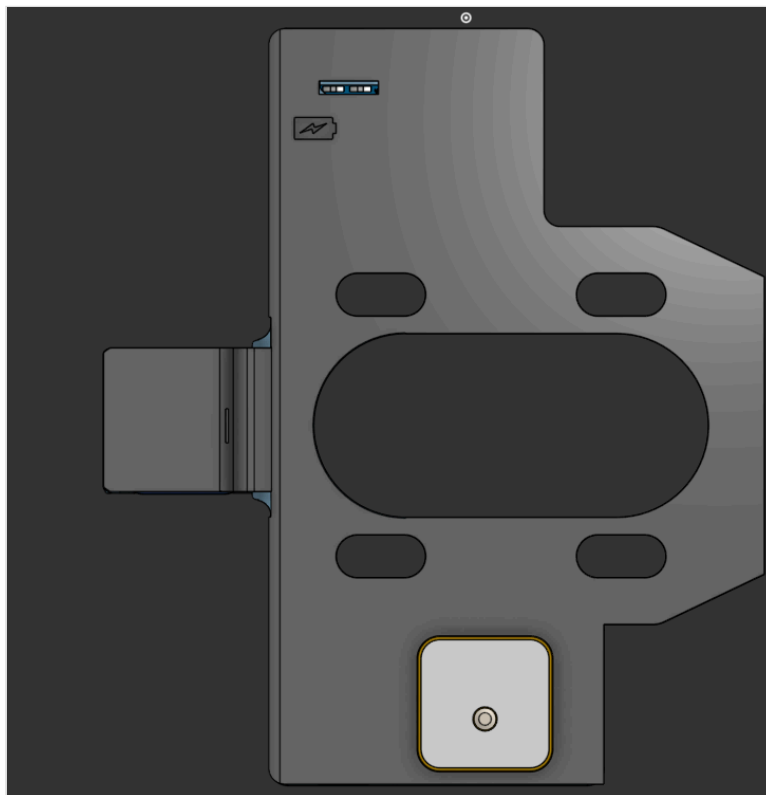
Board enclosure, base and lid separated, showing the IMU pod and the GPS antenna



Board enclosure assembled on its mounting plate



Plan view with the IMU pod, the XIAO, the GPS antenna and the charging modules in place

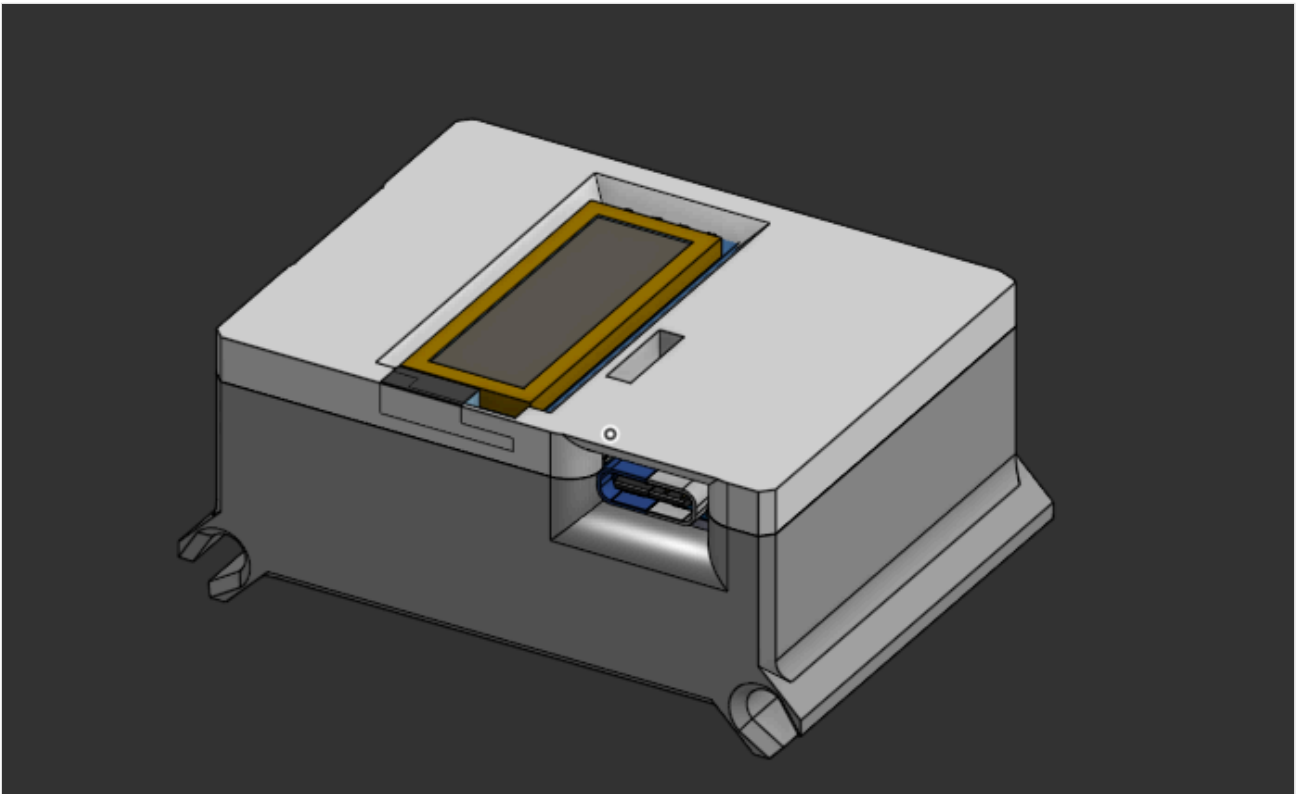


Closed plan view with the status window and charge indicator

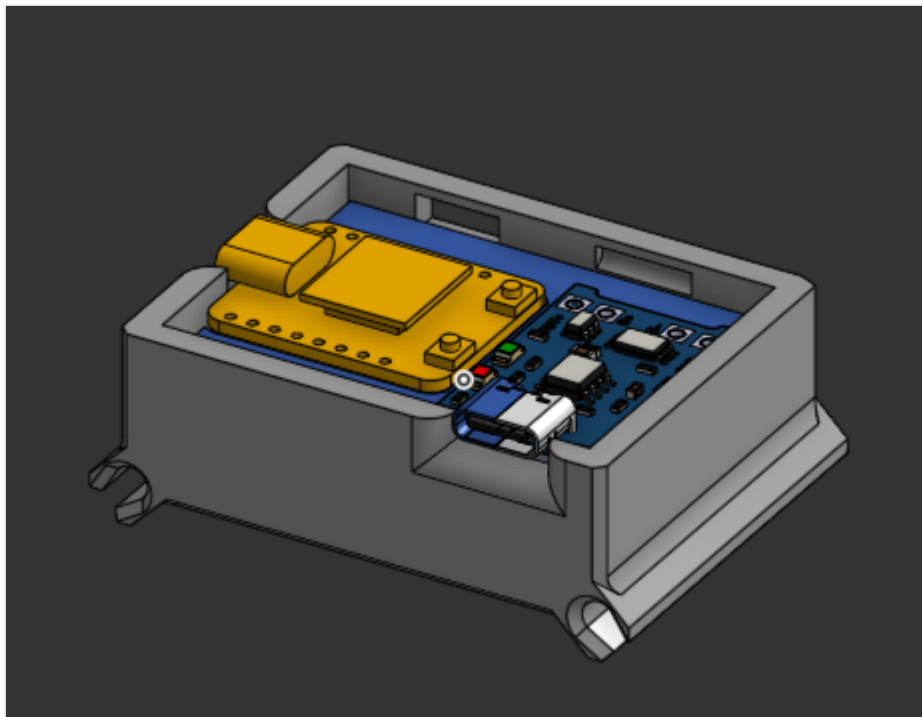
Wrist unit

The wrist enclosure is a lidded box with lugs at all four corners for the strap. The OLED sits in a window in the lid with its four pins (GND, VCC, SCL, SDA) running to the XIAO below, and the two indicator LEDs show

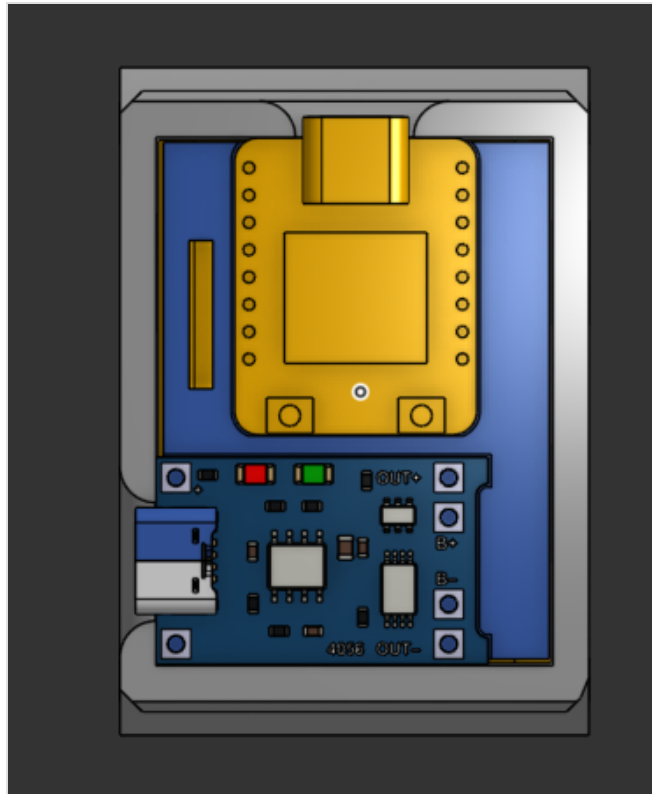
through beneath it. The USB-C port of the charging module is recessed into the side wall, so the unit charges on the wrist without being opened.



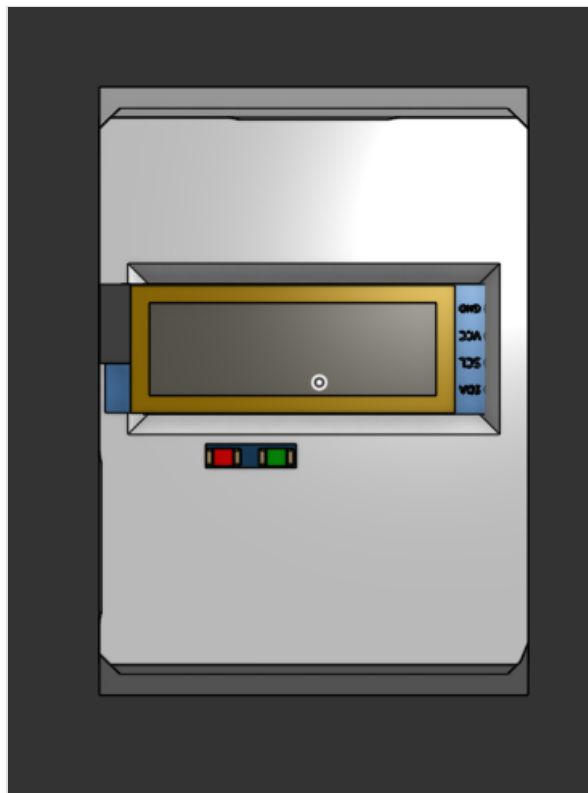
Wrist unit assembled, OLED window in the lid and strap lugs at the corners



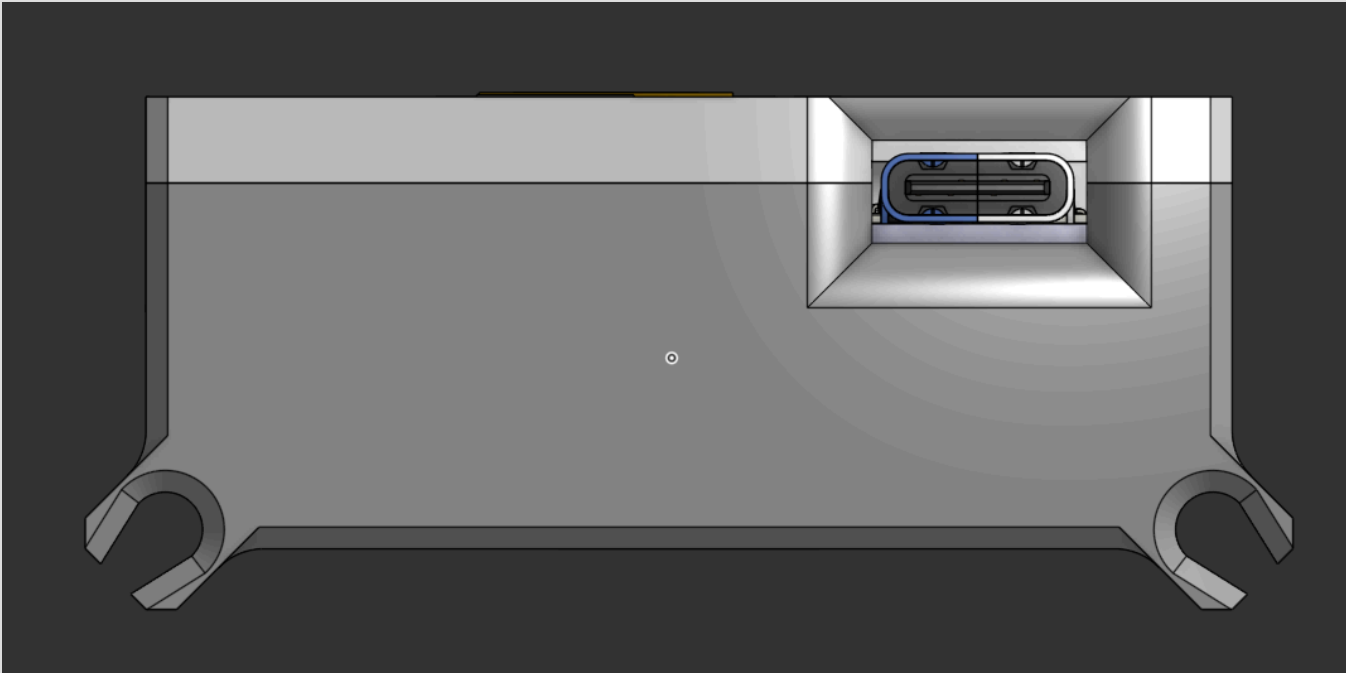
Wrist unit with the lid off, XIAO stacked over the charging module



Interior plan view: XIAO, charging module, indicator LEDs and side button



Front view with the OLED window, its pin labels and the two indicator LEDs



Side view with the recessed USB-C charging port

Open question

Print material, layer height, infill and the finished dimensions of each enclosure are not recorded yet.

Phase 2 — Hardware and Enclosure (Week 2)

Planned: mechanical layout for the IMU and GPS, 3D-printed board housing and wrist casing, ESP32 toolchain, basic IMU and GPS communication.

Delivered: both enclosures printed and populated, the toolchain running on Arduino IDE with ESP32 core 3.2.0 on XIAO_ESP32C3, and both sensors talking to the board.

Hardware as built

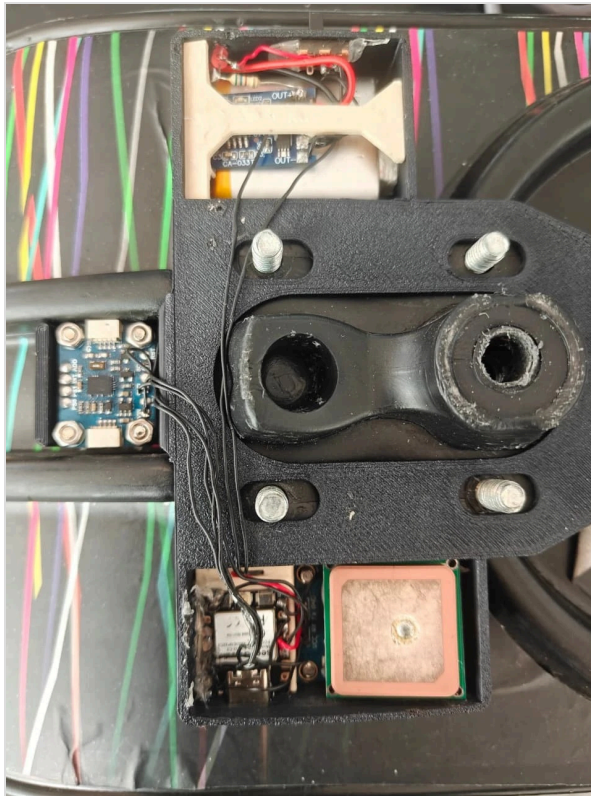
Unit	Component	Connection
Board	Seeed XIAO ESP32-C3	Mounted under the deck
Board	BNO055 IMU	I2C — SDA GPIO6 (D4), SCL GPIO7 (D5), address 0x28, fused mode
Board	NEO-8M GPS	UART1 — RX GPIO20, TX GPIO21, 9600 baud
Board	Battery sense	ADC GPIO4 (ADC1_CH4), divider 30k / 10k
Board	1500 mAh lithium cell, USB-C charging module, slide switch	Cell to charger, charger output through the switch to the XIAO
Wrist	Seeed XIAO ESP32-C3	Worn on the strap
Wrist	SSD1306 128×32 OLED	I2C — SDA GPIO6 (D4), SCL GPIO7 (D5), driven by U8g2
Wrist	Vibration motor	GPIO3
Wrist	Status LED	GPIO8
Wrist	Battery sense	ADC GPIO2 (D0), divider R1 = 30k / R2 = 10k, 11 dB attenuation
Wrist	1500 mAh lithium cell, HW-373 V1.2 USB-C charging module, slide switch	Same arrangement as the board unit
Link	ESP-NOW broadcast	Channel follows the phone hotspot

The BNO055 runs its own sensor fusion, so the ESP32 reads finished Euler angles and a gyroscope vector instead of filtering raw accelerometer data inside the control loop. Battery voltage is averaged over several ADC reads, because a single read on this chip swings the reported percentage by several points.

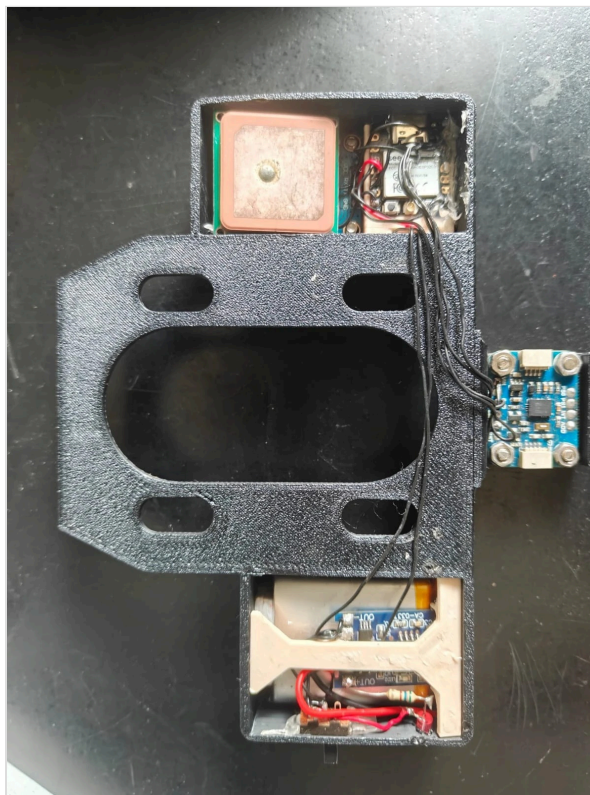
The board sketch uses 91% of the default 1.25 MB program partition and 12% of RAM. Session data lives in a separate LittleFS partition, so a nearly full sketch cannot push the stored rides out.

Printed, populated and mounted

The CAD from Phase 1 came out as two printed enclosures. The plate bolts around the truck baseplate with the IMU pod hanging off the deck's edge; the charging module, cell and power switch sit in the upper compartment, the XIAO and the GPS antenna in the lower one.

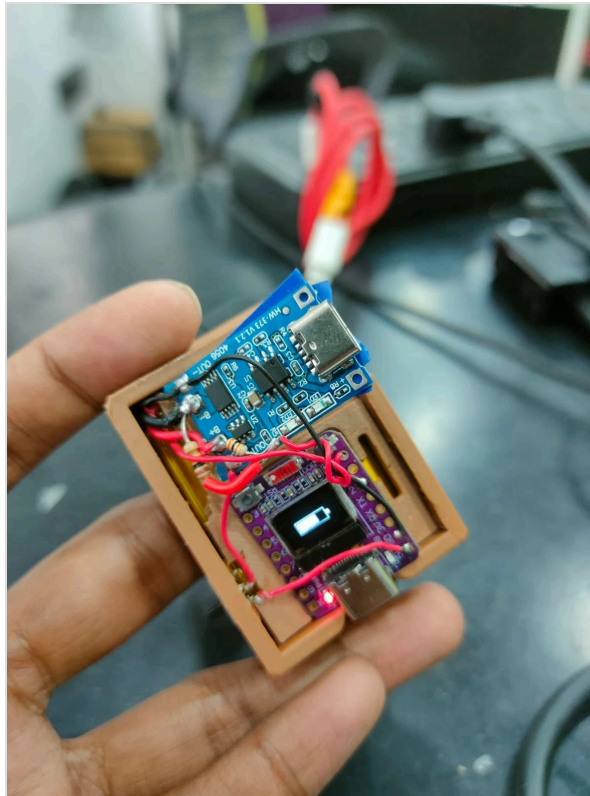


The board unit bolted to the deck around the truck baseplate

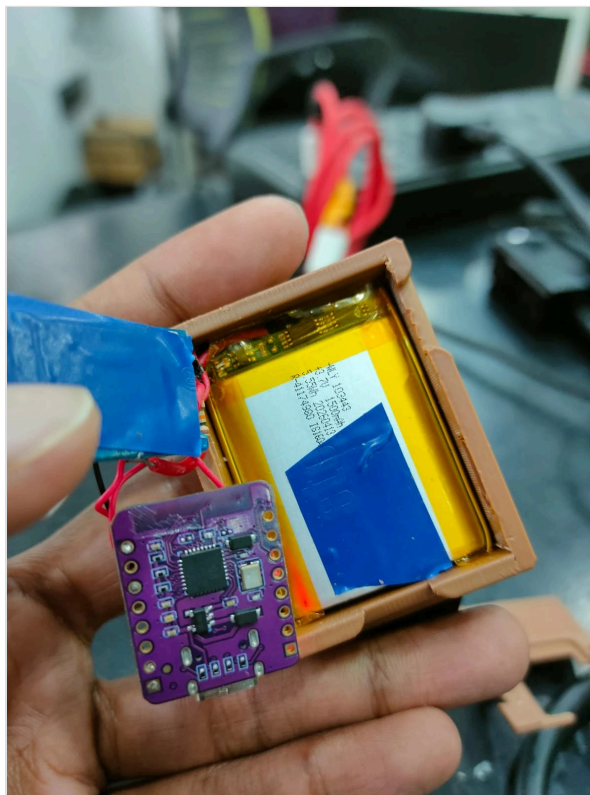


The built plate off the deck: GPS antenna and XIAO, IMU breakout on its pod, charger, cell and switch

The wrist unit stacks an HW-373 V1.2 USB-C charging module (TP4056 class) over the XIAO, with the OLED facing out through the lid.



The built wrist unit: charging module, XIAO and OLED showing the battery icon

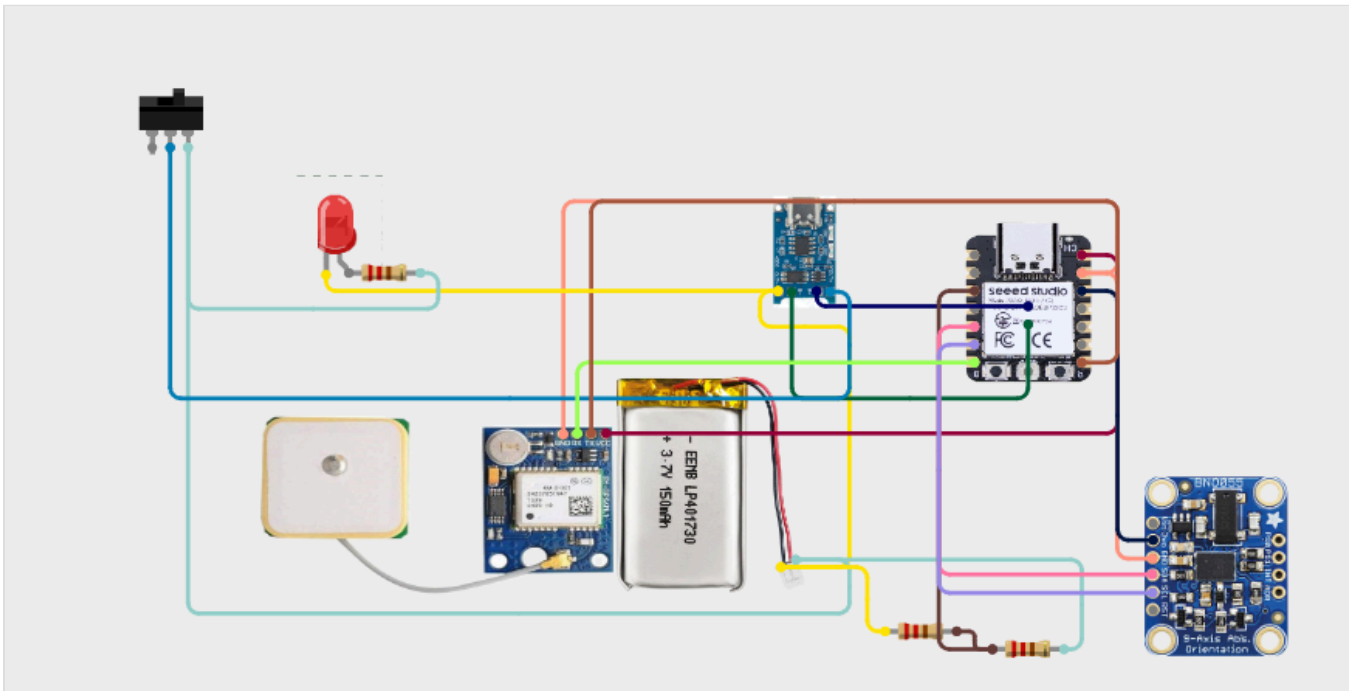


The wrist case with its 1500 mAh cell, lid and board removed

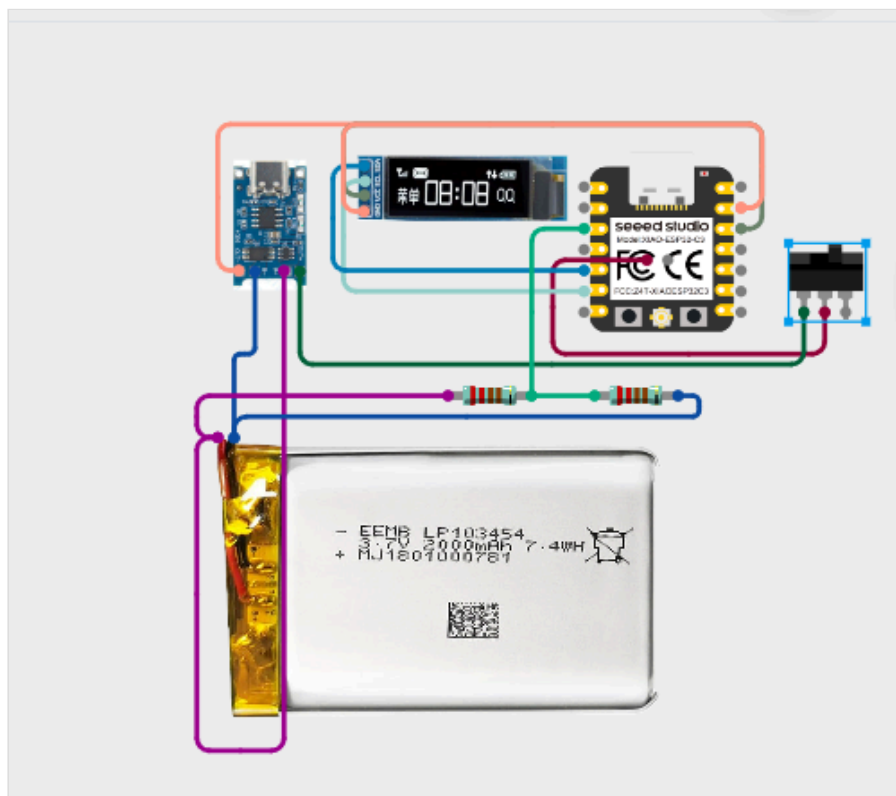
Wiring

Each unit is wired the same way: the cell feeds a USB-C charging module, the module's output passes through a slide switch to the XIAO, and the sensors or display hang off the XIAO's I2C and UART pins. The

resistors are the I2C pull-ups and the indicator-LED series resistor. Both units run a 3.7 V 1500 mAh cell; the capacities printed on the diagram artwork are stand-ins from the drawing tool.



Board unit wiring: switch, indicator LED, USB-C charger, XIAO ESP32-C3, NEO-8M GPS, cell and BNO055



Wrist unit wiring: USB-C charger, 128x32 OLED, XIAO ESP32-C3, slide switch and cell

Signal	Board unit	Wrist unit
I2C data	GPIO6 (D4) to BNO055	GPIO6 (D4) to OLED
I2C clock	GPIO7 (D5) to BNO055	GPIO7 (D5) to OLED
Serial	GPIO20 RX, GPIO21 TX to NEO-8M at 9600 baud	—
Battery sense	GPIO4 (ADC1_CH4) through a 30k/10k divider	GPIO2 (D0) through a 30k/10k divider
Motor	—	GPIO3
Status LED	Indicator LED on the charging module	GPIO8

The firmware checks these rather than assuming them: `imuInit()` scans the I2C bus at boot and prints every address that answers, and `batteryInit()` logs one raw ADC count, so a wrong pad shows up immediately instead of as a reading that never moves.

Phase 3 — Sensor and Embedded System (Week 3)

Planned: firmware reading live IMU and GPS data, the wobble algorithm, and kickflip detection from rotational signatures.

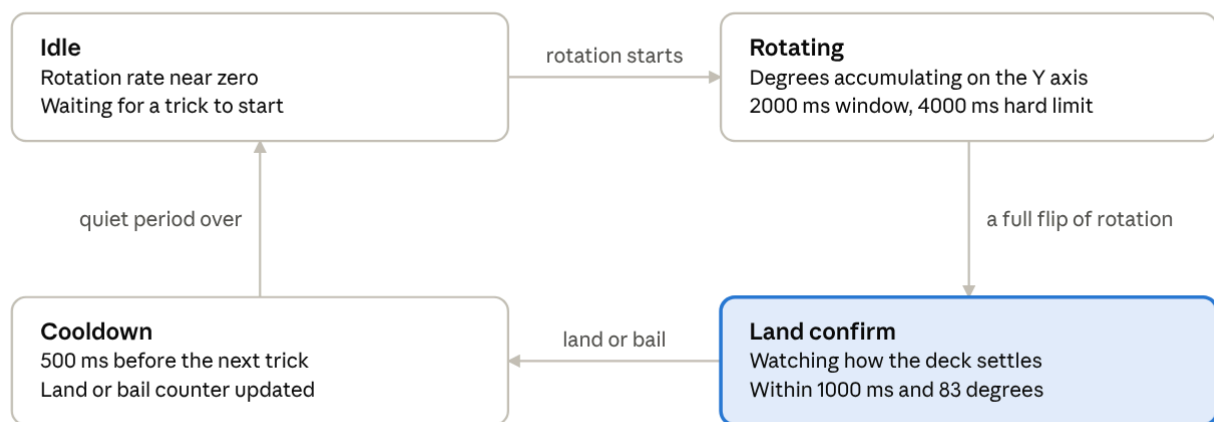
Delivered: a continuous sensor loop, kickflip detection as a four-stage state machine, and wobble detection on counted oscillations with no speed gate.

Detection in one page

Detection is a four-stage state machine over the gyro axis a kickflip rotates around:

Stage	Meaning
IDLE	Rotation rate near zero, waiting for a trick
ROTATING	Degrees accumulating; 2000 ms window, 4000 ms hard limit
LAND_CONFIRM	Watching how the deck settles, within 1000 ms and 83 degrees
COOLDOWN	500 ms quiet period, counter updated

A flip counts as a land only if the deck settles within 83 degrees



The four stages of trick detection

A trick that reaches `LAND_CONFIRM` and settles inside the limit counts as a **land**. One that keeps swinging past 83 degrees, or runs out of time, counts as a **bail**. Speed wobble is separate: three counted oscillations are needed before it is reported, so one swerve does not trigger an alert.

Constant	Value	Purpose
MOTION_WINDOW_MS	2000	How long the active rotation window stays open
MOTION_ABSOLUTE_TIMEOUT_MS	4000	Hard limit on any single trick
MOTION_COOLDOWN_MS	500	Quiet period after a counted trick
LAND_CONFIRM_WINDOW_MS	1000	Time allowed to confirm the landing
LAND_CONFIRM_MAX_DEG	83.0	Swing past which the landing is rejected as a bail
WOBBLE_OSCILLATIONS_REQUIRED	3	Oscillations before a wobble is called

A GPS fix is trusted only when the position is valid, less than 3 seconds old and backed by at least 4 satellites. Speeds under 2 km/h report as zero so a parked board does not drift, and a position step implying more than 20 m/s is rejected as multipath.

The debug page shows the same values live, twice a second, which is what the thresholds were set against.

Debug		×
Stage	IDLE	
Gyro Y	-0.9 deg/s	
Accum Rotation	0.0 deg	
Land Confirm Swing	0.0 deg	
Active Window	0 / 2000 ms	
Absolute Window	0 / 4000 ms	
Cooldown Window	0 / 500 ms	
Oscillations	0 / 3	
Loop Max / Avg	12 / 3.8 ms	
Land Count	1	
Bail Count	0	
Battery	86% (4.02 V)	
GPS Satellites	9	
GPS HDOP	0.9	
GPS Fix Age	0.4 s	
GPS Data In	9426 chars	
GPS Checksum Fails	0	
WiFi	ch 6 -54 dBm	
ESP-NOW	wearable linked	
Free Heap	175 KB	
Uptime	9s	
Download stage_log.csv Clear log		

The debug page: stage, gyro, windows, counts, GPS and radio

Still open

WOBBLE_AXIS and KICKFLIP_AXIS are an assumption about how the IMU sits on this mount. stage_log.csv now records gyro_x and gyro_z beside gyro_y, so one real kickflip shows which axis carries the rotation and settles it from data.

Phase 4 — Wireless Alerts and Bail Detection (Week 4)

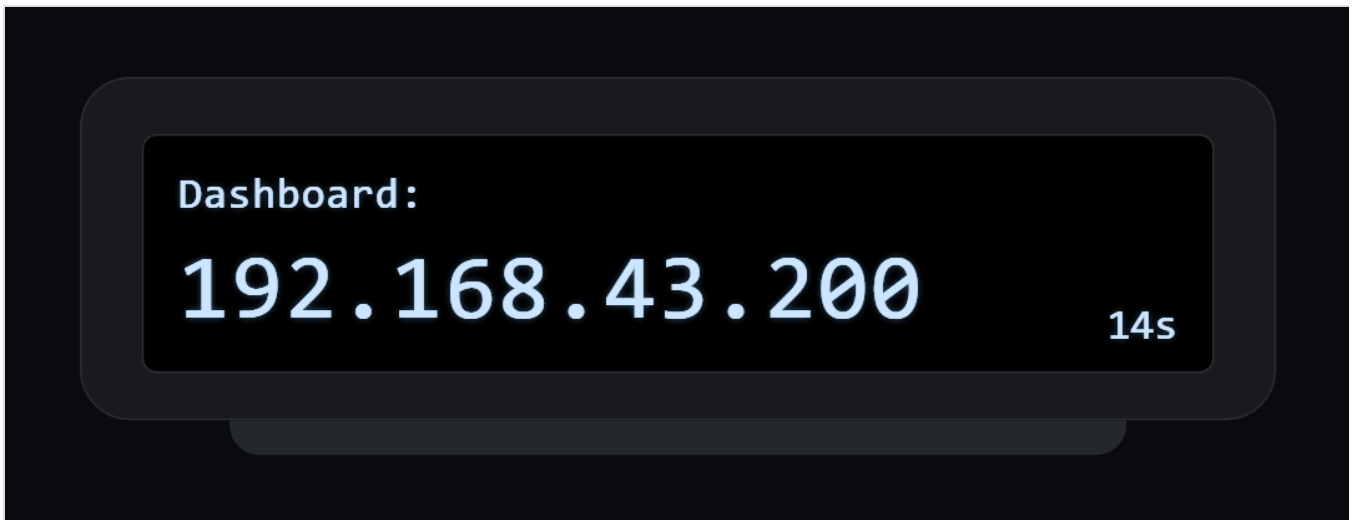
Planned: a wireless link between board and wearable, haptic feedback on confirmed wobbles, and bail detection separating landings from bails.

Delivered: an ESP-NOW link with channel hunting, the wrist motor and LED pulsing on each wobble the board confirms, and bails separated from lands by how far the deck swings after the rotation.

The link

The board broadcasts telemetry over ESP-NOW, which runs alongside its hotspot connection and needs no pairing. Its peer is registered on channel 0, so it follows whatever channel the phone picks. The wrist unit does not know that channel at power-up, so it parks on each channel for 300 ms until packets arrive, then reads the channel the board states inside the packet and retunes to it.

The packet carries speed, distance, the board's battery, its dashboard address, GPS fix state and a `wobbleActive` flag.



The wrist unit showing the dashboard address

Haptic alert path

Step	Where
Three oscillations counted, wobble confirmed	Board firmware, motion state machine
<code>wobbleActive</code> set true for one packet	Board, ESP-NOW telemetry
Flag latched for 500 ms on screen, counted as ongoing for 1500 ms	Wrist firmware
Motor on GPIO3 and LED on GPIO8 pulsed, non-blocking	Wrist firmware
Wobble counted into the recording session	Board, <code>sessionNoteWobble()</code>

Serial printing was moved out of the ESP-NOW receive callback into a drained queue. Printing inside the callback blocks the WiFi task long enough to drop the following packets, which made a healthy radio link

look unreliable.

Bail detection

The plan called for an accelerometer impact spike after a trick. That was replaced: a clean landing can produce a sharp impact and a bail can produce none, so the spike does not separate them. What does is the settling swing — a flip that settles within 83 degrees inside 1000 ms is a land, anything wider or slower is a bail.

Still open

The motor has been fired from bench shakes, not from a real speed wobble at riding speed. That test belongs to Phase 6.

Phase 5 — Companion Dashboard and Data Logging (Week 5)

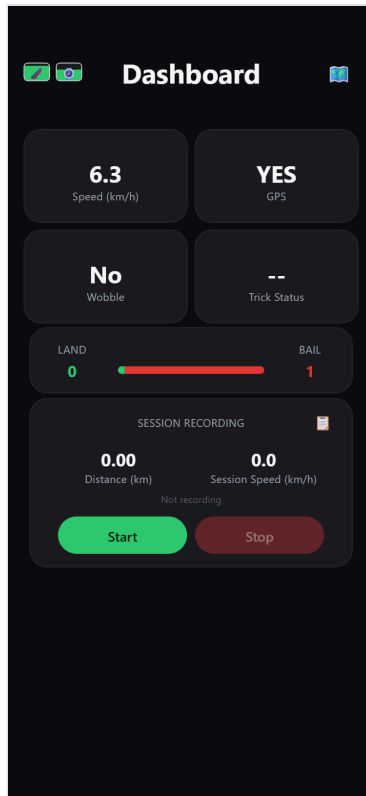
Planned: a companion app or dashboard, GPS route mapping, ride statistics, a post-ride summary and live sync.

Delivered: a dashboard served by the board itself over the phone hotspot, with a live map, session recording, a post-ride summary covering distance, top speed, wobble alerts and trick counts, and per-session downloads.

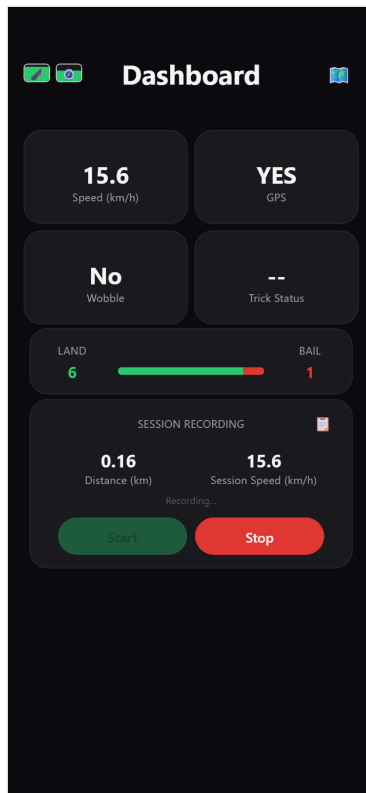
No app to install

The board joins the rider's phone hotspot, takes a DHCP lease, then moves itself to `<subnet>.200` and verifies the move, so the dashboard address is predictable on any phone. It also answers to `skateguard.local`, though phone support for that is inconsistent, which is why the wrist unit displays the numeric address. The page is stored in flash inside the sketch and served whole; the browser polls `/api/live` twice a second.

Endpoint	Purpose
<code>/</code>	The dashboard page
<code>/api/live</code>	Telemetry JSON, polled twice a second
<code>/api/debug</code>	Motion state machine, GPS, radio and memory values
<code>/api/sessions</code>	Saved sessions and their routes
<code>/api/command?cmd=</code>	<code>start</code> , <code>stop</code> , <code>save</code> , <code>discard</code> , <code>delete:<n></code>
<code>/log</code> , <code>/clear</code>	Download or clear the motion log
<code>/manifest.json</code>	Web app manifest for "Add to Home screen"



The dashboard on the phone, waiting to record



Recording in progress

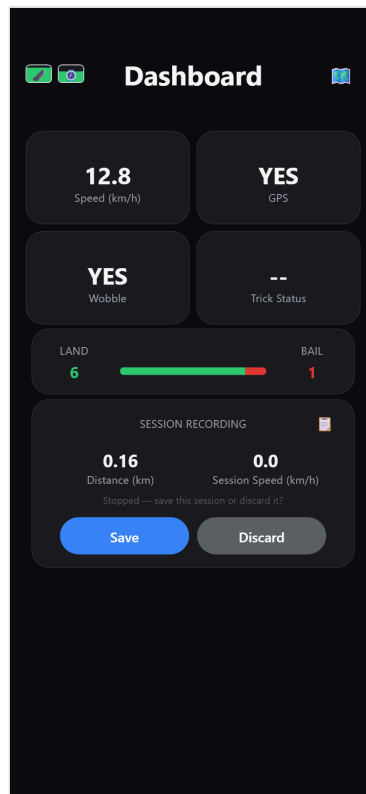
Post-ride summary

Stopping a ride does not save it: the card offers Save or Discard, and nothing is written until one is chosen. A saved ride stores distance, average speed, top speed, land count, bail count and wobble-alert count, with its

route as a separate line.

```
timestamp_ms,avg_speed_kmh,distance_km,land_count,bail_count,top_speed_kmh,wobble_count
842113,11.84,0.874,7,2,19.60,1
```

`top_speed_kmh` and `wobble_count` are appended after the original columns, so a `sessions.csv` written by earlier firmware still parses — the dashboard reads by position and treats the two new fields as unknown when they are missing.



Stopped, offering Save or Discard

 Recorded Sessions (last 7)	
Session 1: 0.874 km • avg 11.84 km/h • L7/B2	 
Session 2: 0.000 km • avg 0.00 km/h • L14/B5	 
Session 3: 1.215 km • avg 13.20 km/h • L9/B3	 
Session 4: 0.163 km • avg 9.11 km/h • L6/B0	 

The session list after saving

The last 7 rides are kept, with a route point every 30 seconds up to 20 points. Each row downloads as a CSV of the ride plus a rendered PNG of its route, drawn in the browser over OpenStreetMap tiles. Route line N belongs to session N, with a  placeholder for rides recorded without GPS.

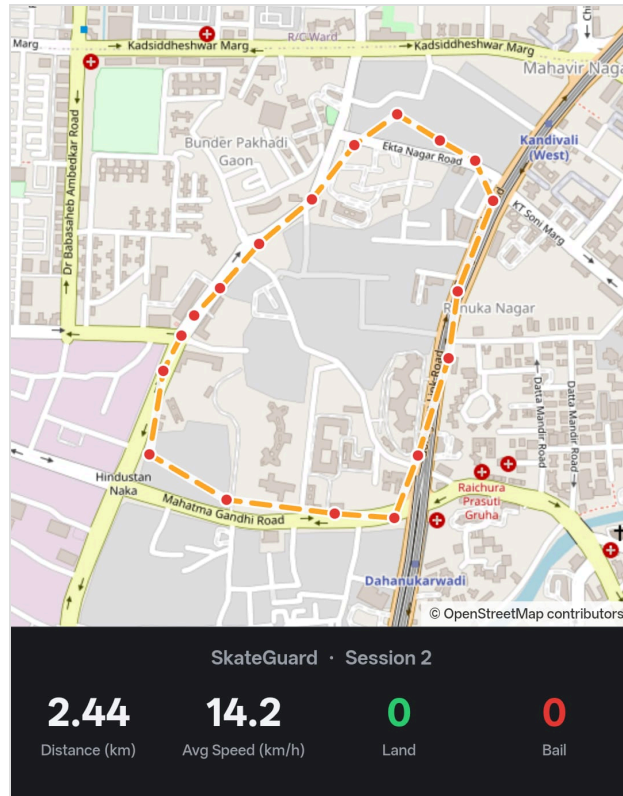
Phase 6 — Testing, Tuning and Demonstration (Week 6)

Planned: full system testing across speeds and surfaces, tuning to cut false triggers, final assembly, demonstration and documentation.

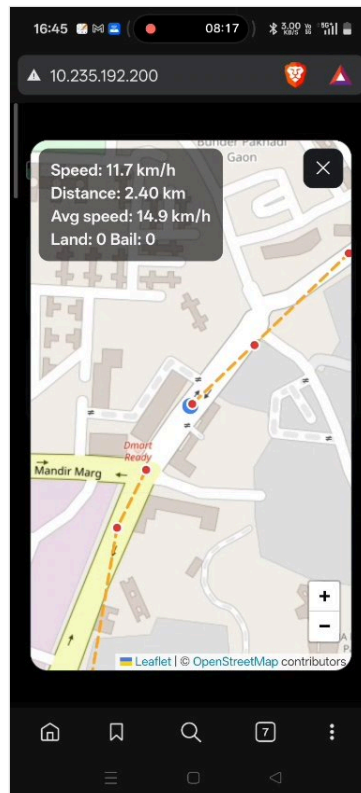
Delivered so far: both units assembled and running, with bench, browser and first field testing complete. On-road tuning and the demonstration are the remaining work.

What has been tested

Area	Method	Result
Trick detection	<code>stage_log.csv</code> , a row every 500 ms holding stage, all three gyro axes, accumulated degrees, each window against its limit and the event that fired	Thresholds set to the Phase 3 values; log downloads cleanly and is capped at 2000 rows
Loop health	Slowest and average loop period recorded and shown on the debug page	Instrumented, because a stalled loop shows up as missed tricks rather than as a timing symptom
GPS	Status line every 5 seconds: satellites, HDOP, characters processed, fix age, checksum failures; first fix after boot timed	Fix acquired outdoors with 4 or more satellites; no fix indoors, as expected
Radio link	Power-up in both orders, walking out of range until <code>NO SIGNAL</code> and back, changing the hotspot so the channel moves	Wrist unit re-finds the board without a reboot in each case
Dashboard	Exercised in a browser against a simulated board: save and discard, the 7-session cap, deleting sessions, downloads with and without a route	All paths work; the route-alignment fault was reproduced and fixed with a no-GPS session saved between two GPS sessions
Field use	Real rides recorded, then opened as live map and session route maps	Routes render over OpenStreetMap tiles and match the ride



A recorded session drawn over OpenStreetMap tiles



The live map during a real ride

Detection accuracy on real rides has not been quantified yet. That is the open item.

What remains

1. Confirm the rotation axis from one real kickflip, comparing gyro_x, gyro_y and gyro_z in `stage_log.csv`, and confirm the I2C and ADC pins from the boot log.
2. Ride with a hand-counted tally of flips, score land and bail counts against it, and tune the thresholds to cut false triggers.
3. Trigger a real speed wobble at riding speed and confirm the wrist motor fires.
4. Decide whether the 25 km/h figure returns as a configurable alert level.
5. Measure battery life on both 1500 mAh units over a full session.
6. Record print material, layer height, infill and enclosure dimensions, then prepare the demonstration.

Changes from the original plan

Four decisions made during the build differ from the Week 1 plan. Each was a change of method, not of objective.

Planned	Built	Reason
MPU6050 IMU	BNO055 IMU	The BNO055 fuses on the sensor and outputs absolute orientation, so rotation is measured directly instead of being filtered on the ESP32 inside the control loop
BLE link to a companion app	ESP-NOW to the wrist unit, dashboard served by the board	A companion app has to be installed and kept in step with the firmware. A page served from the board opens in any phone browser, and ESP-NOW gives a board-to-wrist link with no pairing step
Wobble triggered above 25 km/h	Wobble from three counted oscillations, no speed gate	GPS speed needs 4 satellites and lags real movement, and wobble can start below 25 km/h. The oscillation pattern is the symptom itself, so it is what the detector watches
Bail detected from an accelerometer impact spike	Bail detected from how far the deck swings after the rotation	A clean landing can produce a sharp impact and a bail can produce none, so the spike does not separate them. The settling swing does

The 25 km/h figure is still usable as a reporting threshold rather than a trigger, if the project needs it stated that way, as a configurable alert level rather than a detection gate.

Limitations and next steps

- Detection is tuned for kickflips. Other tricks rotate around different axes and are not classified.
- The dashboard has no login. Anyone on the hotspot can open it, record and delete sessions; the hotspot password is the only barrier.
- A GPS cold start can take several minutes, and nothing is reported until 4 satellites are visible.
- Map tiles come from OpenStreetMap, so the phone needs internet for the map screens. Recording itself does not.
- The sketch fills 91% of the default 1.25 MB partition. Larger features need a different partition scheme, which erases stored sessions.
- Only the last 7 sessions are kept, and a route point every 30 seconds means the drawn line cuts corners on fast laps.
- Deleting a session happens immediately, with no confirmation.

Next steps, in order:

1. Confirm the rotation axis from one real kickflip, comparing the three gyro axes now logged, and confirm the I2C and ADC pins from the boot log.
2. Ride with a hand-counted tally of flips, score land and bail counts against it, and tune the thresholds to cut false triggers.
3. Confirm the wobble oscillation count on a real speed wobble rather than a bench shake.

4. Decide whether the 25 km/h figure returns as an alert level.
5. Battery-life measurement for both units over a full session.
6. Record print material, layer height, infill and enclosure dimensions, then prepare the demonstration.